

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum, Karnataka -590014



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

RAKSHITA KR (1BF24CS243)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum) Department of
Computer Science and Engineering



This is to certify that the Lab work entitled “ANALYSIS AND DESIGN OF ALGORITHMS” carried out by Rakshita KR (1BF24CS1243), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (23CS4PCADA) work prescribed for the said degree.

Jyothi S Nayak
Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

INDEX SHEET

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	1 - 5
2	Implement Johnson Trotter algorithm to generate permutations.	6 - 7
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	8 - 11
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	12 - 15
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	16 - 18
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	19 - 22
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	23 - 26
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	27 - 30
9	Implement Fractional Knapsack using Greedy technique.	31 - 34
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	35 - 37
11	Implement "N-Queens Problem" using Backtracking.	38 - 39

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

LAB PROGRAM 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include    <stdio.h>

#define MAX 100 int

graph[MAX][MAX];

int indegree[MAX];

int queue[MAX];

int front = 0, rear = 0;

void topologicalSort(int
n){    int i, j, count = 0;

for(i = 0; i < n; i++){

    indegree[i] = 0;

    for(i = 0; i < n;

i++){

        for(j = 0; j < n; j++){

if(graph[i][j] == 1){

            indegree[j]++;

        }

    }

}

for(i = 0; i < n; i++){

if(indegree[i] == 0){

    queue[rear++] = i;

    }

}
```

```

    printf("Topological Ordering: ");

while(front < rear){

    int vertex = queue[front++];

    printf("%d ", vertex);

    count++;    for(i = 0; i <

n; i++){

    if(graph[vertex][i] == 1) {

        indegree[i]--;

        if(indegree[i] == 0){

            queue[rear++] = i;

        }

    }

    }

    }    if(count

!= n){

        printf("\nGraph contains a cycle. Topological ordering not possible.");

    }

}

int main(){

    int n, i, j;

    printf("Enter number of vertices: ");

    scanf("%d", &n);    printf("Enter

adjacency matrix:\n");    for(i = 0; i <

n; i++){

        for(j = 0; j < n; j++){

            scanf("%d", &graph[i][j]);

        }

    }

}

```

```

    }

    topologicalSort(n);

    return 0;
}

```

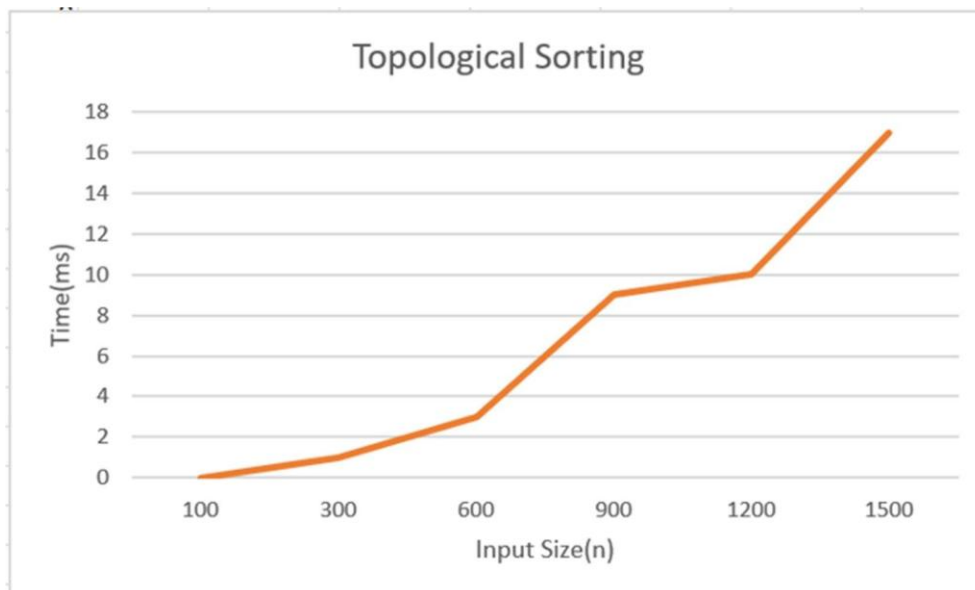
Output

```

Enter number of vertices: 6
Enter adjacency matrix:
0 0 0 0 0 0
0 0 0 1 0 0
1 0 0 0 0 0
1 1 0 0 0 0
0 0 1 1 0 0
0 1 0 0 1 0
Topological Ordering: 5 4 2
Graph contains a cycle. Topological sort not possible.

=== Code Execution Successful ===

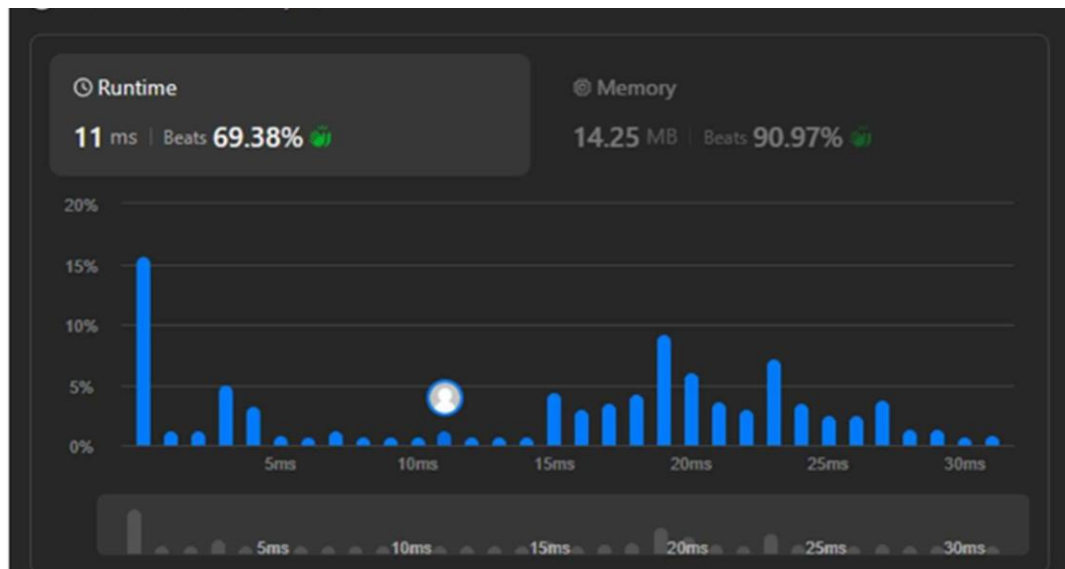
```



LEETCODE 210: COURSE SCHEDULING

```
C v Auto
1 int* findOrder(int numCourses, int** prerequisites, int prerequisitesSize,
2               int* prerequisitesColSize, int* returnSize) {
3
4     int indegree[numCourses], graphSize[numCourses];
5
6     int** graph = malloc(numCourses * sizeof(int*));
7
8     for(int i = 0; i < numCourses; i++) {
9         indegree[i] = 0;
10        graphSize[i] = 0;
11        graph[i] = malloc(numCourses * sizeof(int));
12    }
13
14    for(int i = 0; i < prerequisitesSize; i++) {
15        int a = prerequisites[i][0];
16        int b = prerequisites[i][1];
17
18        graph[b][graphSize[b]++] = a;
19        indegree[a]++;
20    }
21
22    int queue[numCourses];
23    int front = 0, rear = 0;
24
25    for(int i = 0; i < numCourses; i++) {
26        if(indegree[i] == 0)
27            queue[rear++] = i;
28    }
29
30    int* ans = malloc(numCourses * sizeof(int));
31    int idx = 0;
32
33    while(front < rear) {
34        int node = queue[front++];
35
36        ans[idx++] = node;
37
38        for(int i = 0; i < graphSize[node]; i++) {
39            int next = graph[node][i];
40
41            indegree[next]--;
42
43            if(indegree[next] == 0)
44                queue[rear++] = next;
45        }
46    }
47
48    if(idx != numCourses) {
49        *returnSize = 0;
50        return malloc(0);
51    }
52
53    *returnSize = numCourses;
54    return ans;
55 }
```

Accepted Runtime: 0 ms	Accepted Runtime: 0 ms	Accepted Runtime: 0 ms
☒ Case 1 ☒ Case 2 ☒ Case 3	☒ Case 1 ☒ Case 2 ☒ Case 3	☒ Case 1 ☒ Case 2 ☒ Case 3
Input	Input	Input
numCourses = 4	numCourses = 2	numCourses = 1
prerequisites = [[1,0],[2,0],[3,1],[3,2]]	prerequisites = [[1,0]]	prerequisites = []
Output	Output	Output
[0,1,2,3]	[0,1]	[0]
Expected	Expected	Expected
[0,2,1,3]	[0,1]	[0]



LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include<stdio.h> int memory(int p[],int d[],int n){    int m=-
1;    for(int i=0;i<n;i++){        if(d[i]==-1 && i>0 &&
p[i]>p[i-1] && p[i]>m) m=p[i];        if(d[i]==1 && i<n-1
&& p[i]>p[i+1] && p[i]>m) m=p[i];
    }    return
m; } void
main(){
    int n;
    printf("Enter n:");
    scanf("%d",&n);    int
p[n],d[n];    for(int
i=0;i<n;i++){
    d[i]=-1;    p[i]=i+1;
    }

    printf("Permutations:\n");
    while(1){        for(int
i=0;i<n;i++){
    printf("%d ",p[i]);
        }        printf("\n");
    int m=memory(p,d,n);
        if(m==-1) break;
    int pos;        for(int
i=0;i<n;i++){
    if(p[i]==m) pos=i;
```

```

    }

    int newPos=pos+d[pos];

int temp=p[pos];

p[pos]=p[newPos];

p[newPos]=temp;

temp=d[pos];

d[pos]=d[newPos];

d[newPos]=temp;


    for(int i=0;i<n;i++){

if(p[i]>m) d[i]=-d[i];

    }

}

}

```

Output

```

Enter n:3
Permutations:
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

```

```

=== Code Exited With Errors ===

```

LAB PROGRAM 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void merge(int a[], int low, int mid, int
high){    int i = low, j = mid + 1, k = low;

int temp[10000];    while(i <= mid && j <=
high){

        if(a[i] < a[j])

temp[k++] = a[i++];

        else            temp[k++]

= a[j++];

    }

    while(i <= mid)

temp[k++] = a[i++];

while(j <= high)

temp[k++] = a[j++];    for(i

= low; i <= high; i++)

a[i] = temp[i];

} void mergeSort(int a[], int low, int
high){    if(low < high){        int mid =

(low + high) / 2;        mergeSort(a, low,

mid);        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);}
```

```

} int main(){    int a[10000], n, i;

clock_t start, end;    printf("Enter

number of elements: ");    scanf("%d",

&n);    srand(time(NULL));    for(i =

0; i < n; i++)        a[i] = rand() %

1000;    printf("\nGenerated

Array:\n");    for(i = 0; i < n; i++)

printf("%d ", a[i]);    start = clock();

mergeSort(a, 0, n - 1);    end =

clock();    printf("\n\nSorted

Array:\n");    for(i = 0; i < n; i++)

printf("%d ", a[i]);

    double time_taken = (double)(end - start) /

CLOCKS_PER_SEC;    printf("\n\nTime taken = %f seconds\n",

time_taken);    return 0;

}

```

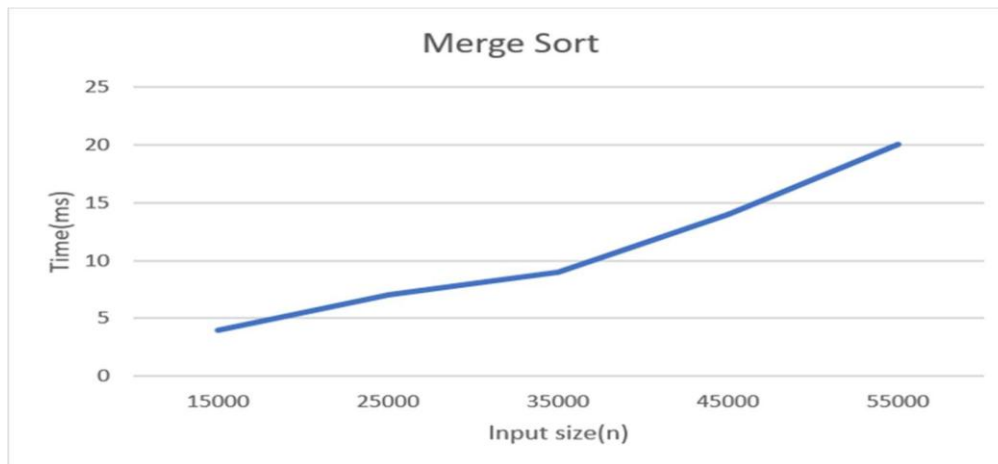
Output

```

Enter the size of array: 10
Enter elements: 1 4 3 2 77 85 23 32 44 67
Sorted array: 1 2 3 4 23 32 44 67 77 85

=== Code Execution Successful ===

```



LEETCODE 26: REMOVE DUPLICATE ELEMENTS

```
C v Auto
1  #include <stdio.h>
2
3  int removeDuplicates(int nums[], int n)
4  {
5      if(n == 0)
6          return 0;
7
8      int i, k = 1;
9
10     for(i = 1; i < n; i++)
11     {
12         if(nums[i] != nums[i - 1])
13         {
14             nums[k] = nums[i];
15             k++;
16         }
17     }
18
19     return k;
20 }
21
22
```

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Expected

[1,2]

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

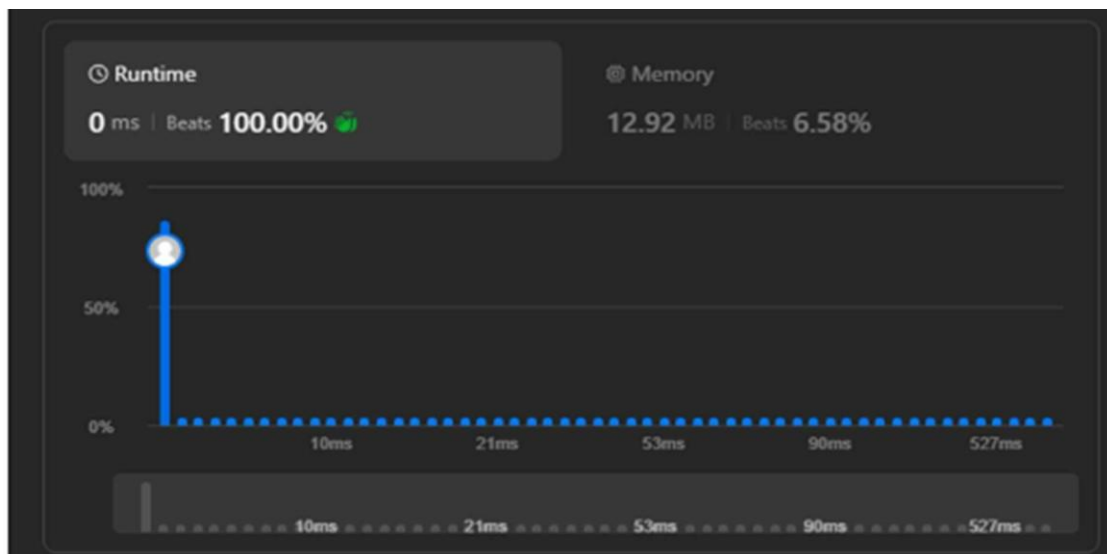
nums =
[0,0,1,1,1,2,2,3,3,4]

Output

[0,1,2,3,4]

Expected

[0,1,2,3,4]



LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h> void

swap(int *a, int *b) {

int temp = *a;  *a = *b;

    *b = temp;

} int partition(int arr[], int low, int high)

{    int pivot = arr[low];    int i = low +

1;    int j = high;    while (1) {

while (i <= high && arr[i] <= pivot)

        i++;    while

(arr[j] > pivot)

        j--;    if (i < j)

swap(&arr[i], &arr[j]);    else

{        swap(&arr[low],

&arr[j]);        return j;}

    }

} void quickSort(int arr[], int low, int

high) {    if (low < high) {        int p =

partition(arr, low, high);

quickSort(arr, low, p - 1);

quickSort(arr, p + 1, high);

    }

} int main()

{
```

```

int n;

printf("Enter number of elements: ");

scanf("%d", &n);

int arr[n];    srand(time(0));

printf("Random Numbers:\n");

for (int i = 0; i < n; i++) {

arr[i] = rand() % 100;

printf("%d ", arr[i]);

}    clock_t start, end;    start =

clock();    quickSort(arr, 0, n - 1);

end = clock();

printf("\n\nSorted Numbers:\n");

for (int i = 0; i < n; i++) {

printf("%d ", arr[i]);

}

double time_taken = ((double)(end - start)) /

CLOCKS_PER_SEC;    printf("\n\nTime taken: %f seconds\n",

time_taken);    return 0;}

```

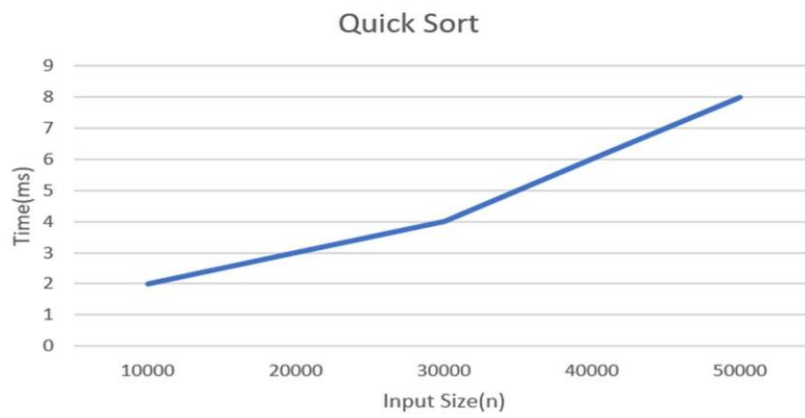
Output

```

Enter number of elements:7
Enter elements:5 6 855 75 56 2 3
Sorted array:2 3 5 6 56 75 855
Execution time:0.00200 ms

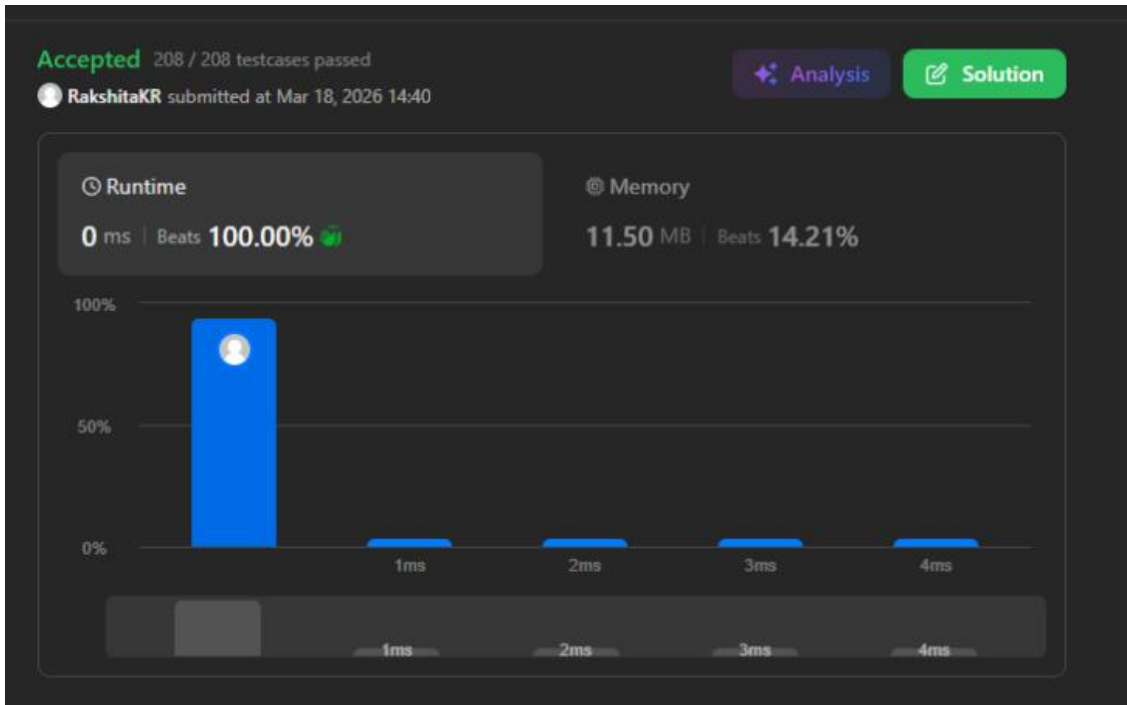
=== Code Exited With Errors ===

```

LEETCODE 21: MERGE TWO LISTS

```
1  /**
2   * Definition for singly-linked list.
3   * struct ListNode {
4   *     int val;
5   *     struct ListNode *next;
6   * };
7   */
8
9  struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2) {
10
11     struct ListNode dummy; // dummy node
12     struct ListNode* tail = &dummy; // pointer to build result
13
14     dummy.next = NULL;
15
16     while (list1 != NULL && list2 != NULL) {
17         if (list1->val <= list2->val) {
18             tail->next = list1;
19             list1 = list1->next;
20         } else {
21             tail->next = list2;
22             list2 = list2->next;
23         }
24         tail = tail->next;
25     }
26
27     // Attach remaining nodes
28     if (list1 != NULL)
29         tail->next = list1;
30     else
31         tail->next = list2;
32
33     return dummy.next;
34 }
35
```



Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

list1 =
[1, 2, 4]

list2 =
[1, 3, 4]

Output

[1, 1, 2, 3, 4, 4]

Expected

[1, 1, 2, 3, 4, 4]

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

list1 =
[]

list2 =
[]

Output

[]

Expected

[]

LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>

#include<time.h>

#include<stdlib.h> void

heapify(int H[],int n){   for(int

i=n/2;i>=1;i--){       int k=i;

int v=H[i];       int heap=0;

while(heap==0 && 2*k<=n){

    int j=2*k;       if(j<n

&& H[j]<H[j+1])

        j=j+1;

    if(v>=H[j])

        heap=1;       else{

        H[k]=H[j];

        k=j;

        }

    }

    H[k]=v;

    }

} void heapSort(int H[],int

n){   heapify(H,n);

for(int i=n;i>=2;i--){

    int temp=H[1];

    H[1]=H[i];       H[i]=temp;

    int k=1;       int v=H[1];

    while(2*k<=i-1){       int
```

```

j=2*k;          if(j<i-1 &&
H[j]<H[j+1])
                j=j+1;

if(v>=H[j])

break;

H[k]=H[j];

k=j;    }

    H[k]=v;

    }

} void

main(){

    int n;

    printf("Enter number of

elements:");   scanf("%d",&n);   int

H[n+1];   printf("Enter the

elements:\n");   for(int

i=1;i<=n;i++){

scanf("%d",&H[i]);

    }

    clock_t start=clock();

heapSort(H,n);

    clock_t end=clock();   double cpu_time_used=((double)(end-

start))/CLOCKS_PER_SEC)*1000.0;   printf("\nElements after heap sort:");

for(int i=1;i<=n;i++){   printf("%d ",H[i]);

    }

    printf("\nInput Size:%d Execution Time:%.5f",n,cpu_time_used);

}

```

Output

Enter number of elements:5

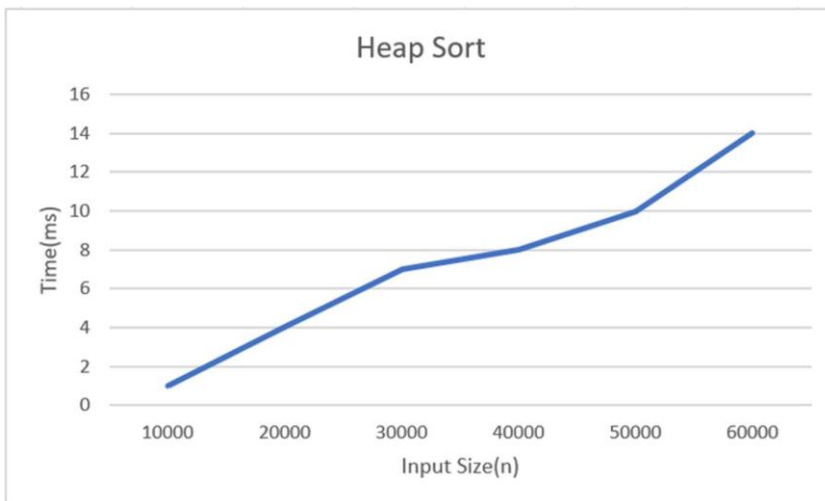
Enter the elements:

4 10 3 5 1

Elements after heap sort:1 3 4 5 10

Input Size:5 Execution Time:0.00200

=== Code Exited With Errors ===



LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include<stdio.h>

int max(int a,int b){

return (a>b)?a:b;

} void

main(){

    int n,m;    printf("Enter the number of objects and
capacity:");    scanf("%d %d",&n,&m);    int

v[n+1][m+1];    int p[n+1],w[n+1],x[n+1];

printf("Enter the weight and profit:\n");    for(int

i=1;i<=n;i++){        scanf("%d %d",&w[i],&p[i]);

    }

    for(int i=0;i<=n;i++){

x[i]=0;

    }

    for(int i=0;i<=n;i++){        for(int j=0;j<=m;j++){

if(i==0 || j==0) v[i][j]=0;        else if(w[i]<=j) v[i][j]=max(v[i-

1][j],p[i]+v[i-1][j-w[i]]);        else v[i][j]=v[i-1][j];

    }

}    int

i=n;

    int j=m;

while(i>0 && j>0){

    if(v[i][j]!=v[i-

1][j]){        x[i]=1;
```

```

j=j-w[i];    }    i=i-
1;
}

printf("Selected objects weight and
profit:\n");    printf("%-5s %-5s","w","p");

for(int i=0;i<=n;i++){    if(x[i]==1)

printf("\n%-5d %-5d",w[i],p[i]);

}

printf("\nTotal profit:%d",v[n][m]);
}

```

Output

```

Enter the number of objects and capacity:4 7
Enter the weight and profit:
1 1
3 4
4 5
5 7
Selected objects weight and profit:
w      p
3      4
4      5
Total profit:9

=== Code Exited With Errors ===

```

LEETCODE 638: SHOPPING OFFERS

```

1  #include <limits.h>
2
3  int min(int a, int b) { return a < b ? a : b; }
4  int dfs(int* price, int** special, int specialSize,
5         int* needs, int n) {
6
7      int cost = 0;
8
9      for (int i = 0; i < n; i++)
10         cost += price[i] * needs[i];
11
12     for (int i = 0; i < specialSize; i++) {
13         int newNeeds[n];
14         int valid = 1;
15
16         for (int j = 0; j < n; j++) {
17             if (special[i][j] > needs[j]) {
18                 valid = 0;
19                 break;
20             }
21             newNeeds[j] = needs[j] - special[i][j];
22         }
23
24         if (valid) {
25             int offer = special[i][n];
26             cost = min(cost, offer +
27                      dfs(price, special, specialSize, newNeeds, n));
28         }
29     }
30
31     return cost;
32 }
33
34 int shoppingOffers(int* price, int priceSize,
35                   int** special, int specialSize,
36                   int* specialColSize,
37                   int* needs, int needsSize) {
38
39     return dfs(price, special, specialSize, needs, needsSize);
40 }

```


Accepted Runtime: 0 ms

☒ Case 1
 ☒ Case 2

Input

price =
[2,5]

special =
[[3,0,5],[1,2,10]]

needs =
[3,2]

Output

14

Expected

14

Accepted Runtime: 0 ms

☒ Case 1
 ☒ Case 2

Input

price =
[2,3,4]

special =
[[1,1,0,4],[2,2,1,9]]

needs =
[1,2,1]

Output

11

Expected

11

Accepted 65 / 65 testcases passed

RakshitaKR submitted at May 06, 2026 14:59

[Analysis](#)
[Solution](#)

Runtime

146 ms | Beat 13.04%

Memory

305.30 MB | Beat 13.04%

Code | C

```

1 int min(int a, int b) {
2     return a < b ? a : b;
3 }
4
5
6 int dfs(int* price, int priceSize, int** special, int specialSize, int* need
7 int key = 0;
8 for (int i = 0; i < needsSize; i++) {

```

[View more](#)

LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>

#include<limits.h>

void main(){

    int n;    printf("Enter the number of vertices:");

    scanf("%d",&n);    int a[n][n];    printf("Enter the
weights and 0 for no edges:\n");    for(int
i=0;i<n;i++){        for(int j=0;j<n;j++)

            {

                scanf("%d",&a[i][j]);

            }

        }

    for(int i=0;i<n;i++){        for(int
j=0;j<n;j++){            if(i!=j && a[i][j]==0)
a[i][j]=INT_MAX;

        }

    }

    for(int i=0;i<n;i++){        for(int j=0;j<n;j++){

        for(int k=0;k<n;k++){            if(a[i][k]!=INT_MAX
&& a[k][j]!=INT_MAX){

                if(a[i][j]>a[i][k]+a[k][j]) a[i][j]=a[i][k]+a[k][j];

            }

        }

    }

    }    printf("Shortest pairs:\n");    for(int
i=0;i<n;i++){        for(int j=0;j<n;j++){
```

```

if(a[i][j]==INT_MAX) printf("Infinity ");

else printf("%d ",a[i][j]);

    }

printf("\n");

    }

}

```

Output

```

Enter the number of vertices:4
Enter the weights and 0 for no edges:
0 5 0 10
0 0 3 0
0 0 0 1
0 0 0 0
Shortest pairs:
0 5 8 9
Infinity 0 3 4
Infinity Infinity 0 1
Infinity Infinity Infinity 0

=== Code Exited With Errors ===

```

LEETCODE 1334: CITY WITH SMALLEST NUMBER OF NEIGHBOURS

```

C v Auto
1  #include <limits.h>
2
3  #define INF 1000000000
4
5  int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {
6      int dist[101][101];
7
8      // Initialize distance matrix
9      for (int i = 0; i < n; i++) {
10         for (int j = 0; j < n; j++) {
11             if (i == j)
12                 dist[i][j] = 0;
13             else
14                 dist[i][j] = INF;
15         }
16     }
17
18     // Fill edges
19     for (int i = 0; i < edgesSize; i++) {
20         int u = edges[i][0];
21         int v = edges[i][1];
22         int w = edges[i][2];
23         dist[u][v] = w;
24         dist[v][u] = w;
25     }
26
27     // Floyd-Marshall
28     for (int k = 0; k < n; k++) {
29         for (int i = 0; i < n; i++) {
30             for (int j = 0; j < n; j++) {
31                 if (dist[i][k] + dist[k][j] < dist[i][j]) {
32                     dist[i][j] = dist[i][k] + dist[k][j];
33                 }
34             }
35         }
36     }
37
38     int minReachable = INT_MAX;
39     int resultCity = -1;
40
41     // Count reachable cities
42     for (int i = 0; i < n; i++) {
43         for (int j = 0; j < n; j++) {
44             if (i != j && dist[i][j] <= distanceThreshold) {
45                 count++;
46             }
47         }
48
49         // Choose minimum count, break tie by larger index
50         if (count <= minReachable) {
51             minReachable = count;
52             resultCity = i;
53         }
54     }
55
56     return resultCity;
57 }
58

```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n =
4

edges =
[[0, 1, 3], [1, 2, 1], [1, 3, 4], [2, 3, 1]]

distanceThreshold =
4

Output

3

Expected

3

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n =
5

edges =
[[0, 1, 2], [0, 4, 8], [1, 2, 3], [1, 4, 2], [2, 3, 1], [3, 4, 1]]

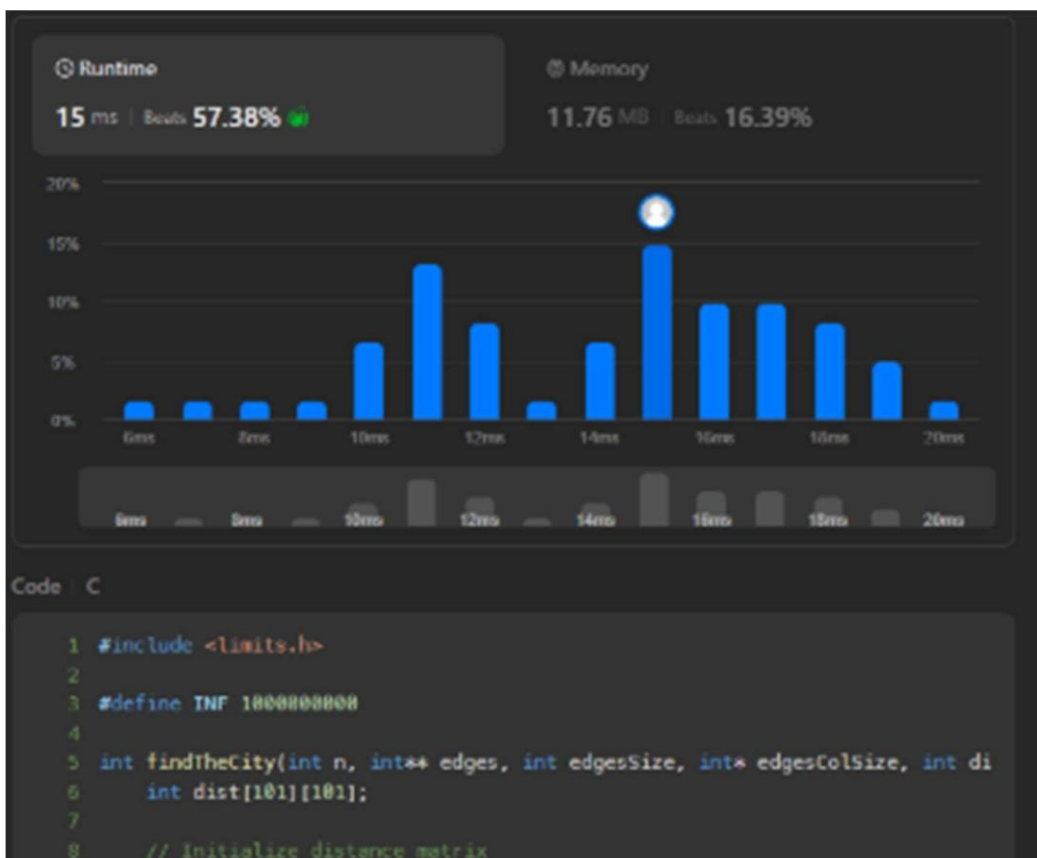
distanceThreshold =
2

Output

0

Expected

0



8a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

27

```

    }

    }

    }

    printf("%d - %d = %d\n", a, b,
min);    minCost += min;

visited[b] = 1;    edges++;

    }

    printf("Minimum Cost = %d\n", minCost);

    return 0;

}

```

Output

```

Enter number of vertices: 4
Enter adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
Edges in MST:
0 - 3 = 5
3 - 2 = 4
0 - 1 = 10
Minimum Cost = 19

=== Code Execution Successful ===

```

8b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```

#include <stdio.h>

#define MAX 10 #define

INF 999 int parent[MAX];

int find(int x){

while(parent[x] != x)

```

```

x = parent[x];    return x; }

void unionSet(int a, int b){

parent[find(a)] = find(b);

} int main(){    int

cost[MAX][MAX];

    int n, i, j;    int min, a, b;    int

edges = 0, minCost = 0;

printf("Enter number of vertices: ");

scanf("%d", &n);    printf("Enter

adjacency matrix:\n");    for(i = 0; i <

n; i++){        for(j = 0; j < n; j++){

scanf("%d", &cost[i][j]);

if(cost[i][j] == 0)            cost[i][j]

= INF;

        }    }    for(i = 0;

i < n; i++){

parent[i] = i;

    }

    printf("Edges in MST:\n");

while(edges < n - 1){        min

= INF;        for(i = 0; i < n;

i++){            for(j = i + 1; j <

n; j++){                if(cost[i][j]

< min){                    min =

cost[i][j];                    a = i;

b = j;

                }

            }

        }

```



```

        }    if(find(a) != find(b)){
printf("%d - %d = %d\n", a, b, min);

minCost += min;    unionSet(a, b);

edges++;

        }

        cost[a][b] = cost[b][a] = INF;

    }

printf("Minimum Cost = %d\n", minCost);

return 0;
}

```

Output

```

Enter number of vertices: 4
Enter adjacency matrix:
0 10 6 5
10 0 0 5
6 0 0 4
5 15 4 0
Edges in MST:
2 - 3 = 4
0 - 3 = 5
1 - 3 = 5
Minimum Cost = 14

=== Code Execution Successful ===

```

LAB PROGRAM 9:

Implement fractional knapsack problem using Greedy technique.

```
#include<stdio.h> struct object{

float profit;   float weight;

float pw;}; void sort(struct

object o[],int n){   struct object

temp;   for(int i=0;i<n-1;i++) {

for(int j=0;j<n-i-1;j++){

if(o[j].pw<o[j+1].pw){

temp=o[j];

o[j]=o[j+1];

o[j+1]=temp;}

    }

    }

} int

main(){

    int n;   printf("Enter number of

objects:");   scanf("%d",&n);   struct

object o[n];   printf("Enter the profit and

weight for:\n");   for(int i=0;i<n;i++){

printf("Object %d:",i);

        scanf("%f

%f",&o[i].profit,&o[i].weight);

o[i].pw=o[i].profit/o[i].weight;}   sort(o,n);

int size;   printf("Enter the size of the

knapsack:");   scanf("%d",&size);   float

total_profit=0;   int rm=size;   int i=0;
```

```

while(rm!=0){    if(o[i].weight<rm){

total_profit+=o[i].profit;    rm-

=o[i].weight;

    }    else{

total_profit+=((o[i].profit*rm)/o[i].weight);

rm=0;    }    i++;

    }

printf("%-12s %-12s %-12s %-12s\n", "Object", "Profit", "Weight",

"Ratio");    for(int i = 0; i < n; i++){    printf("%-12d %-12.2f %-12.2f %-

12.2f\n",

        i, o[i].profit, o[i].weight, o[i].pw);

    }

printf("\nMaximum profit: %.2f\n", total_profit);

return 0;

}

```

Output

```

Enter number of objects:3
Enter the profit and weight for:
Object 0:60 10
Object 1:100 20
Object 2:120 30
Enter the size of the knapsack:50
Object      Profit      Weight      Ratio
0           60.00       10.00       6.00
1           100.00      20.00       5.00
2           120.00      30.00       4.00

Maximum profit: 240.00

=== Code Execution Successful ===

```

LEETCODE 322: COIN CHANGE

```

C  Auto
1  int coinChange(int* coins, int coinsSize, int amount) {
2      int dp[amount + 1];
3
4      // Initialize dp array
5      for (int i = 0; i <= amount; i++) {
6          dp[i] = amount + 1; // large value (infinity)
7      }
8
9      dp[0] = 0; // base case
10
11     // Fill dp array
12     for (int i = 1; i <= amount; i++) {
13         for (int j = 0; j < coinsSize; j++) {
14             if (coins[j] <= i) {
15                 if (dp[i - coins[j]] + 1 < dp[i]) {
16                     dp[i] = dp[i - coins[j]] + 1;
17                 }
18             }
19         }
20     }
21
22     // If not possible
23     if (dp[amount] > amount) {
24         return -1;
25     }
26
27     return dp[amount];
28 }

```

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

coins =
[1,2,5]

amount =
11

Output

3

Expected

3

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

coins =
[2]

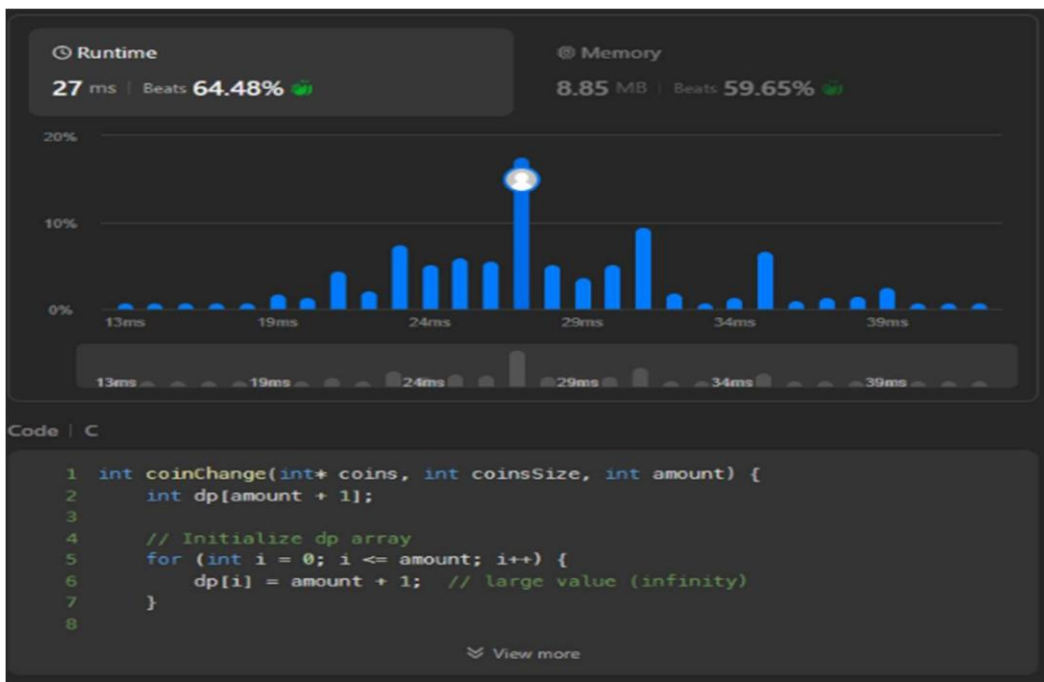
amount =
3

Output

-1

Expected

-1



LAB PROGRAM 10

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>

#include<limits.h> void

p(int parent[],int j){

if(parent[j]==j){

printf(" Path:%d ",j);

return;

}

p(parent,parent[j]);

printf("%d ",j);

} void

main(){

    int n;    printf("Enter the number of vertices:");

scanf("%d",&n);    int a[n][n];    printf("Enter the

weights and enter 0 for no edges:\n");    for(int

i=0;i<n;i++){        for(int j=0;j<n;j++){

scanf("%d",&a[i][j]);

        }    }    int

distance[n];

int visited[n];

int parent[n];

int source;

printf("Enter the

source vertex:");

scanf("%d",&so

urce);    for(int
```

```

i=0;i<n;i++){

distance[i]=(i!=s

ource &&

a[source][i]==0)

?INT_MAX:a[so

urce][i];

parent[i]=source;

visited[i]=0;

}

distance[source]=0;

visited[source]=1;  for(int

i=1;i<n;i++){      int

min=INT_MAX;

    int u=-1;      for(int j=0;j<n;j++){

if(!visited[j] && distance[j]<min){

min=distance[j];      u=j;

    }

    }

    if(u==-1) break;      visited[u]=1;

for(int v=0;v<n;v++){      if(a[u][v]!=0

&& !visited[v]){

if(distance[u]+a[u][v]<distance[v]){

distance[v]=distance[u]+a[u][v];

parent[v]=u;

    }

    }

    }

}

```

```

    printf("The distances are:\n");    for(int
i=0;i<n;i++){        printf("%d to %d",source,i);

if(distance[i]==INT_MAX) printf(":No path");

        else{            p(parent,i);

printf(":%d",distance[i]);

        }

printf("\n");

    }

}

```

Output

```

Enter the number of vertices:5
Enter the weights and enter 0 for no edges:
0 10 0 5 0
0 0 1 2 0
0 0 0 0 4
0 3 9 0 2
7 0 6 0 0
Enter the source vertex:0
The distances are:
0 to 0 Path:0 :0
0 to 1 Path:0 3 1 :8
0 to 2 Path:0 3 1 2 :9
0 to 3 Path:0 3 :5
0 to 4 Path:0 3 4 :7

=== Code Exited With Errors ===

```


LAB PROGRAM 11

Implement “N-Queens Problem” using Backtracking.

```
#include<stdio.h> #include<stdlib.h> int
x[20]; int counter=1; int place(int k,int i); void
nqueens(int k,int n){   for(int i=1;i<=n;i++){
if(place(k,i)){        x[k]=i;        if(k==n){
printf("\nSolution:%d\n",counter++);
for(int j=1;j<=n;j++){          for(int
k=1;k<=n;k++){          if(x[j]==k)
printf("Q ");          else printf(". ");
        }
printf("\n");
        }
printf("\n");
        }      else{
nqueens(k+1,n);
        }
    }
}
int place(int k,int i){   for(int j=1;j<=k;j++){
if(x[j]==i || (abs(j-k)==abs(x[j]-i))) return 0;
    }   return
1; } void
main(){
    int n;
```

```
printf("Enter number of queens:");  
  
scanf("%d",&n);  nqueens(1,n);  
  
if(counter==1) printf("No solution");  
  
}
```

Output

Enter number of queens:4

Solution:1

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

Solution:2

```
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

=== Code Exited With Errors ===